



中国科学院大学

University of Chinese Academy of Sciences

研究生学位论文中期报告

报告题目 OAuth 协议的实现安全性分析

学生姓名 朱正轩 学号 2023E8015082070

指导教师 徐静 职称 研究员

学位类别 电子信息硕士

学科专业 软件工程

研究方向 网络空间安全

培养单位 中国科学院软件研究所

填表日期 2025 年 12 月

中国科学院大学制

报告提纲

一 选题的背景及意义	1
1.1 选题背景	1
1.2 选题意义	2
二 国内外本学科领域的发展现状与趋势	3
2.1 OAuth 实现中的常见漏洞	4
2.2 OAuth 参数级安全机制在复杂调用链中的局限性	6
三 研究内容和预期目标	8
3.1 研究内容	8
3.2 预期目标	9
四 研究进展和阶段性成果	9
4.1 OAuth 实现安全分析的阶段性研究成果	10
4.2 MCP 原生授权流程安全问题的分析进展	10
4.3 MCP-AuthZ Gateway 的协议与架构设计成果	10
4.4 MCP-Auth Gateway 的工程实现进展	11
4.5 阶段性评估设计与后续实现方向	11
4.6 阶段性成果总结	12
五 研究工作计划与进度安排	12

一 选题的背景及意义

1.1 选题背景

随着云计算与互联网服务的快速发展，第三方应用对用户数据和资源的受控访问需求日益增长。OAuth 2.0 协议作为当前最广泛采用的授权框架，为不同主体之间的访问授权提供了一种去中心化、可扩展的解决方案，已被主流互联网平台和 API 服务广泛部署，并逐渐成为事实上的行业标准。

然而，与其广泛应用形成鲜明对比的是，OAuth 在现实系统中的安全问题长期高发。大量研究与实际安全事件表明，OAuth 并非一个“自动保证安全”的协议，其安全性高度依赖于实现细节与使用方式。一旦在参数生成、传递、校验或授权流程设计上存在偏差，攻击者即可利用协议灵活性所带来的空间实施多种攻击。

已有研究系统揭示了 OAuth 实现中常见的安全隐患，包括但不限于：授权码被截获或重放、state 参数缺失或验证不当导致的登录跨站请求伪造（Login CSRF）、redirect-uri 校验不严格引发的授权码泄露、客户端混淆（Client Confusion）与混淆代理（Confused Deputy）攻击，以及访问令牌作为 Bearer Token 被窃取后可被任意重放使用等问题。这类攻击并非协议理论层面的缺陷，而是源于 OAuth 在工程实践中被错误组合、错误部署或错误信任。

更为重要的是，OAuth 在设计之初主要面向的是传统 Web 或移动应用场景，其典型信任模型假设授权发起方、令牌使用方与资源访问方之间的关系相对清晰。然而，随着系统架构向微服务化、代理化和自动化方向发展，OAuth 的使用环境正变得愈发复杂。在多跳代理、跨服务调用以及自动决策系统中，原有的信任边界逐渐模糊，OAuth 授权上下文被“拉长”，攻击面随之显著扩大。

近年来，大语言模型（Large Language Model, LLM）及其工具调用能力的兴起，进一步加剧了这一问题。以 Model Context Protocol (MCP) 为代表的新型协议，通过标准化消息格式，使模型能够自动调用外部工具和资源，极大拓展了模型的应用场景。然而，在现有 MCP 设计中，OAuth 被直接引入作为外部资源访问的授权机制，而未对授权发起、令牌流转及权限约束进行细化建模。这种“直接复用 OAuth”的做法，在一定程度上继承并放大了 OAuth 在复杂系统中已被验证的安全风险。

因此，在 MCP 这一新兴应用场景下，如何正视 OAuth 在现实实现中存在的安全问题，并对其进行有针对性的协议层增强，而非简单复用传统实践，成为一个具有现实紧

迫性和研究价值的问题。这也构成了本文进一步研究 OAuth 在 MCP 中应用安全性及其改进方案的核心背景。

1.2 选题意义

1.2.1 OAuth 协议的普及性与基础性

OAuth 2.0 协议是当前部署最广泛的授权与单点登录 (Single Sign-On, SSO) 协议之一，同时也是新一代身份认证标准 OpenID Connect 的基础。凭借其灵活的授权模型与良好的扩展性，OAuth 已被广泛应用于各类在线服务，成为现代网络安全基础设施的重要组成部分。

在实际应用中，OAuth 被广泛用于支持第三方应用与主流互联网平台之间的安全集成，使用户能够在不直接暴露凭证的前提下，将其资源的有限访问权限授予外部应用。这一特性在电子商务、社交媒体、教育平台以及医疗健康系统等领域尤为重要。例如，在电子商务场景中，OAuth 被用于支持支付、物流与用户账户系统之间的安全协作；在社交媒体平台中，OAuth 支撑了跨平台登录与内容共享；在教育与医疗领域，OAuth 为敏感数据的受控访问提供了基础保障 [1]。

OAuth 在这些关键领域的广泛应用，充分体现了其在保障授权安全、提升用户体验以及促进系统互操作性方面的重要价值。然而，正是由于 OAuth 在现实系统中的高度普及，其安全性问题一旦出现，往往会对大规模用户和关键基础设施造成严重影响。

1.2.2 OAuth 实现层面的安全漏洞与现实风险

尽管 OAuth 已成为事实上的授权标准协议，但大量研究表明，其安全性在很大程度上取决于具体实现方式，而非协议规范本身的抽象设计。现实系统中，OAuth 的实现往往存在不同程度的偏差和简化，这为攻击者提供了可利用的空间。

已有工作系统性地揭示了 OAuth 实现中普遍存在的安全漏洞，包括但不限于：授权码拦截与重放、state 参数缺失或校验不当导致的登录跨站请求伪造 (Login CSRF)、redirect_uri 校验不严格引发的授权码泄露、客户端混淆攻击以及混淆代理 (Confused Deputy) 攻击等 [2-4]。这些攻击往往并非源于协议理论缺陷，而是由于实现方未能严格遵循推荐的安全实践，或在复杂系统环境中错误地组合和使用 OAuth 机制 [3]。

随着网络生态系统的不断演进，OAuth 的应用环境也日益复杂。在物联网、智能城市以及跨服务自动化系统中，授权请求链路被显著拉长，原有清晰的信任边界逐渐模

糊，从而引入了新的攻击面 [4, 5]。此外，现实中多起高曝光度的安全事件进一步表明，即便是成熟的大型平台，其 OAuth 实现也可能存在严重漏洞。例如，微软和 Facebook 等知名服务提供商均曾因 OAuth 实现缺陷导致大规模访问令牌泄露或账户被接管 [6, 7]。

这些事实表明，单纯依赖协议规范或开发者自觉，难以从根本上保证 OAuth 在复杂应用场景下的安全性。

1.2.3 从“安全实现”到“体系化授权管理”的必要性

确保 OAuth 的安全实施对于保护用户数据和维护在线服务的可信性至关重要。现有研究通常通过总结安全最佳实践、引入额外防护机制（如 PKCE、多因素认证、令牌绑定等）来提升 OAuth 实现的安全性 [8–11]。这些方法在一定程度上缓解了特定攻击面，但仍高度依赖实现方的正确部署和持续维护。

然而，OAuth 本质上是一个多方参与的授权协议，其安全性需要资源所有者、客户端应用、授权服务器以及资源服务器等多方共同保障。一旦授权体系中的关键组件（尤其是授权服务器或代理组件）出现实现缺陷，往往会对整个生态系统产生影响。现实攻击事件表明，即使在大型服务提供商中，OAuth 实现错误依然难以完全避免，这使得“如何对 OAuth 的使用过程进行系统化约束和治理”成为一个亟需解决的问题。

在新兴的模型驱动系统与工具调用框架中，这一问题尤为突出。以 Model Context Protocol (MCP) 为代表的协议通过引入中间代理和自动化决策机制，进一步拉长了 OAuth 授权链路，使得传统以客户端或授权服务器为中心的安全假设不再充分。在此背景下，仅强调 OAuth 的“安全实现”已难以满足需求，有必要在协议架构层面引入统一的授权控制与安全增强机制。

因此，研究如何通过引入专门的授权治理组件（如 Gateway），对 OAuth 授权流程进行集中约束、能力划分与安全增强，不仅具有重要的工程实践价值，也为 OAuth 在新型应用场景中的安全演进提供了新的研究方向。这正是本文在后续工作中进一步探讨 OAuth 在 MCP 场景下安全应用问题，并提出 MCP-AuthZ 架构的核心动机所在。

二 国内外本学科领域的发展现状与趋势

随着 OAuth 2.0 协议在各类互联网服务中的广泛部署，其安全性问题逐渐成为学术界和工业界的研究重点。已有大量研究从形式化分析、实证测量以及现实攻击案例等多个角度，对 OAuth 协议在实际实现中存在的安全风险进行了系统探讨。这些研究表

明，OAuth 的安全问题并非单一漏洞所致，而是源于协议高度灵活、参与方众多以及实现复杂度较高等因素的综合结果。

Bansal 等最早从实际系统出发，分析了 OAuth 在真实部署中存在的安全漏洞，指出了授权流程中多个参数被错误使用或忽略所引入的攻击面 [2]。Fett 等随后在 CCS 上对 OAuth 2.0 进行了系统性的形式化建模与验证，揭示了协议设计与实现假设之间的偏差，并指出多类此前未被充分认识的攻击路径 [3]。此后，多项工作从不同维度扩展了这一研究方向，包括对 OAuth 实现的实证安全测量 [12, 13]、对新型攻击向量的挖掘 [14, 15]，以及对 OAuth 安全增强机制的综合评估 [4, 16]。

总体来看，现有研究逐步形成了一种共识：OAuth 的安全风险往往可以追溯到具体协议参数在实现和使用过程中的失当处理。因此，从 OAuth 协议关键参数出发，对攻击类型进行系统分类，已成为分析 OAuth 安全性的重要研究路径。

2.1 OAuth 实现中的常见漏洞

2.1.1 基于 state 参数的 CSRF 攻击

在 OAuth 实现中，跨站请求伪造（Cross-Site Request Forgery, CSRF）是最早被发现且至今仍然高发的一类安全漏洞。该攻击利用用户浏览器中已建立的认证状态，诱导用户在不知情的情况下向授权或登录端点发起恶意请求，从而破坏授权上下文的一致性 [2, 12]。

针对 OAuth 场景，研究者进一步指出，CSRF 攻击的根本原因在于客户端未正确使用或校验 state 参数。该参数本应作为请求与响应之间的绑定标识，用于防止跨请求注入。然而，在大量实际部署中，state 参数要么缺失，要么未与用户会话进行严格绑定，从而使攻击者能够重放或伪造授权响应 [17]。

在此基础上，Arshad 等系统研究了 OAuth 登录流程中的 Login CSRF 攻击 [14]。在该攻击中，攻击者利用 OAuth 登录端点，将自身账户的授权结果注入受害者浏览器，使受害者在不知情的情况下登录到攻击者的账户。尽管该攻击不直接窃取受害者账户，但其后果可能更加隐蔽且严重，例如诱导受害者在攻击者账户下上传敏感数据或执行关键操作 [17–19]。

这类攻击表明，OAuth 中请求级参数缺乏统一、强制的校验与绑定机制，单纯依赖客户端实现正确性难以从根本上消除风险。这一问题在后续的复杂系统和多跳代理场景中尤为突出。

2.1.2 基于授权码的拦截与重放攻击

授权码拦截攻击（Authorization Code Interception Attack）是 OAuth 2.0 授权码模式中最具代表性的攻击之一。攻击者通过监听、篡改或劫持授权响应中的重定向过程，获取授权服务器发放的授权码，并在随后伪装合法客户端完成令牌交换，从而非法获取访问令牌。

Fett 等在其形式化分析中指出，OAuth 标准并未强制要求将授权码与客户端实例、设备环境或会话上下文进行绑定，从而使攻击者能够在代码转发路径中实现拦截与重放 [3]。Li 和 Mitchell 的实证研究进一步表明，现实系统中常见的 open redirect、跳转中继页面以及第三方脚本注入，均可被用于实施授权码拦截 [18]。

为缓解该攻击，OAuth 社区提出了 PKCE（Proof Key for Code Exchange）机制，通过在授权请求与令牌交换之间引入额外的绑定参数，显著提升了公共客户端场景下的安全性 [20]。OAuth 2.1 草案也明确将 PKCE 作为授权码模式的强制要求 [21]。

然而，近期研究指出，PKCE 在实际部署中仍可能因错误配置、兼容性回退机制或实现缺陷而被绕过 [15]，表明仅通过单一机制强化某一参数，仍不足以在复杂系统中提供全面防护。

2.1.3 基于 redirect_uri 的攻击

重定向 URI 校验不严格是 OAuth 实现中被反复验证的高危漏洞之一。Fett 等指出，若授权服务器在校验 redirect_uri 时允许模糊匹配或部分匹配，将直接导致授权码泄露风险 [3]。Li 和 Mitchell 的测量研究发现，大量 OAuth 服务提供商仍允许使用前缀匹配、通配符域名或动态参数拼接，从而为攻击者注册恶意 URI 提供空间 [18]。

Innocenti 等进一步揭示了 redirect URI 截断和视觉混淆攻击路径，攻击者可通过构造特定 URI 绕过校验逻辑，将授权结果重定向至其控制的终端 [15]。尽管 OAuth 2.1 草案和主流平台已明确禁止此类宽松校验策略 [21, 22]，但现实系统中仍频繁发现绕过与配置错误 [19]。

该类攻击再次表明，OAuth 参数校验逻辑分散在不同实现中，缺乏统一的强制约束机制，导致即便已有明确安全建议，仍难以在生态层面彻底消除风险。

2.1.4 客户端冒充与混淆代理问题

客户端冒充攻击是 OAuth 实现中另一类重要安全问题。攻击者通过伪造或滥用合法客户端的 `client_id`，诱导用户完成授权流程，从而非法获取访问令牌 [23, 24]。在某些授权模式下，客户端身份验证机制较弱，授权服务器无法有效区分真实客户端实例与攻击者构造的请求。

进一步地，研究者指出，在多服务、多代理环境中，OAuth 容易演化为混淆代理 (Confused Deputy) 问题，即某一受信任组件在不具备完整上下文的情况下，被诱导使用其合法权限为攻击者服务。这一问题在引入中间代理、自动化调用或跨系统授权时尤为突出。

随着 OAuth 被逐步引入模型驱动系统和工具调用框架，其客户端与资源访问方之间的边界进一步模糊，传统以“客户端”为中心的安全假设面临新的挑战。这一趋势也促使研究者开始关注如何在协议架构层面引入统一的授权约束与安全控制点，而非仅依赖单个参数或实现细节的修补。

2.2 OAuth 参数级安全机制在复杂调用链中的局限性

现有关于 OAuth 安全性的研究，大多围绕协议关键参数（如 `state`、`redirect_uri`、`authorization code`、`access token` 等）的正确使用与验证展开，并通过引入附加绑定机制（如 PKCE、Token Binding、DPoP）来缓解特定攻击向量。这类研究在传统 Web 或移动应用场景下取得了显著成效，有效降低了 OAuth 在单一客户端—授权服务器—资源服务器模型中的安全风险。

然而，随着系统架构逐渐向多层代理化、服务编排化以及自动化决策方向演进，OAuth 参数级安全机制在复杂调用链中的适用性开始受到挑战。在现实系统中，授权请求往往不再由最终资源使用方直接发起，访问令牌的使用路径也可能跨越多个中间组件。此时，OAuth 原有以“授权服务器”为中心、以“客户端”为令牌使用方的安全假设逐渐弱化，参数级绑定机制难以覆盖完整的授权上下文。

具体而言，OAuth 参数级安全机制主要面临以下局限性：

第一，授权上下文被拉长且被分割。在多跳调用场景中，授权请求的发起者、授权结果的接收者以及访问令牌的实际使用者可能并非同一实体。尽管 PKCE 等机制能够在授权请求与令牌交换阶段建立绑定，但一旦访问令牌被交付给中间代理组件，其后续使用行为往往脱离了最初的绑定语义。这使得参数级防护难以约束令牌在整个生命周

期内的使用边界。

第二，安全机制高度依赖各参与方实现的一致性。OAuth 的安全增强机制通常以“建议”或“最佳实践”的形式存在，其正确性依赖客户端、授权服务器和资源服务器的协同实现。然而，在实际部署中，不同组件往往由不同组织或团队维护，安全能力和实现水平参差不齐，导致部分参数校验和绑定逻辑被省略、弱化或错误实现，从而重新引入攻击面。

第三，缺乏统一的授权治理与策略执行点。现有 OAuth 机制主要关注单次授权流程的正确性，而对授权结果的使用缺乏集中约束。令牌一旦被签发，资源服务器通常仅依据令牌本身的有效性与 scope 字段进行判断，而无法获知该令牌是否符合更高层次的安全策略，例如最小权限原则、使用场景限制或调用风险等级等。这使得 OAuth 难以在复杂系统中实现细粒度、可审计的授权治理。

上述问题在新兴的模型驱动系统中表现得尤为突出。近年来，大语言模型（Large Language Model, LLM）逐步具备了调用外部工具和服务的能力，用以完成搜索、数据查询、业务操作等任务。为支持模型与工具之间的标准化交互，Model Context Protocol (MCP) 被提出作为一种通用协议框架，用于描述工具能力、参数结构及调用结果。

在 MCP 的现有设计中，OAuth 被直接引入作为外部资源访问的授权机制 [25]。然而，与传统 OAuth 应用不同，MCP 场景下的授权流程通常涉及模型、Host、Client、Server 以及外部资源等多个参与方，且工具调用可能由模型自动触发。这种架构显著增加了授权链路的复杂性，使得访问令牌更容易被中间组件持有和转发，从而放大了 OAuth 在现实实现中已被验证的安全问题。

近期已有研究和工程实践开始注意到 LLM 工具调用场景中的授权风险。一些工作指出，模型驱动的自动化调用可能引入新的混淆代理风险和权限扩散问题，现有基于 OAuth 的授权模式难以为模型行为提供足够的安全约束 [26, 27]。工业界的相关白皮书和安全指南也逐步强调，需要在模型与外部资源之间引入额外的安全控制层，以避免访问令牌被滥用或超范围使用 [28, 29]。

总体而言，现有 OAuth 研究在参数级安全机制方面已取得大量成果，但这些机制在复杂调用链和模型驱动系统中的适用性仍存在明显不足。这一现状表明，有必要在保持 OAuth 协议兼容性的前提下，引入一种能够集中处理授权、约束令牌使用、并统一执行安全策略的架构性机制，从而弥补参数级防护在新型应用场景中的局限性。这也为后续引入以 Gateway 为核心的 MCP-AuthZ 授权增强架构奠定了研究基础。

三 研究内容和预期目标

围绕 OAuth 协议在复杂系统中的安全问题，特别是其在 Model Context Protocol (MCP) 等新型应用场景下的授权风险，本文拟从协议参数安全分析与授权架构设计两个层面展开研究。

3.1 研究内容

3.1.1 基于协议参数的 OAuth 实现安全问题分析

首先，本文将从 OAuth 2.0 协议的关键参数出发，对现实系统中已被发现的 OAuth 安全攻击进行系统化分类与分析。与以往仅从攻击现象或漏洞类型进行归纳的研究不同，本文将重点关注以下问题：

- OAuth 协议中各关键参数（如 state、redirect_uri、authorization code、access token、client_id 等）在协议流程中承担的安全语义；
- 各类攻击如何利用参数生成、传递或校验阶段的实现缺陷破坏授权上下文的一致性；
- 不同攻击之间在根本成因上的共性，以及这些问题在现实部署中的高发原因。

通过系统梳理国内外相关文献与真实安全事件，本文将总结 OAuth 实现中常见的安全风险模式，分析现有安全增强机制（如 PKCE、严格重定向校验等）在缓解特定攻击方面的有效性及其局限性，为后续研究提供安全约束基础。

3.1.2 OAuth 在 MCP 授权模型中的问题分析

在上述分析基础上，本文将以 Model Context Protocol (MCP) 为具体研究对象，对其原生授权模型进行梳理和分析。重点研究 MCP 在引入 OAuth 作为外部资源授权机制时存在的潜在问题，包括：

- MCP 授权流程中各参与方的角色划分与信任边界；
- OAuth 访问令牌在 MCP 调用链中的流转方式及其安全影响；
- 原生 MCP 设计中在权限最小化、令牌约束、撤销与审计等方面的不足。

通过分析 MCP 授权流程与 OAuth 安全假设之间的不匹配，本文将进一步明确在 MCP 场景下引入额外授权控制机制的必要性。

3.1.3 基于 Gateway 的 MCP-AuthZ 授权增强架构设计

针对 MCP 原生授权模型的不足，本文拟提出一种以 Gateway 为核心的 MCP-AuthZ 授权增强架构。该架构在保持 MCP 协议兼容性的前提下，引入统一的授权控制点，用于集中处理 OAuth 授权相关的安全逻辑。主要研究内容包括：

- Gateway 在 MCP 架构中的位置与功能边界设计；
- 工具能力声明与授权约束的结构化表达方式；
- 授权流程中能力分级、权限协商与安全策略执行机制；
- Gateway 在令牌使用约束、撤销与审计中的作用。

通过该设计，本文旨在探索一种适用于 MCP 及类似的新型系统架构，将 OAuth 安全最佳实践协议化、体系化的方法。

3.2 预期目标

围绕上述研究内容，本文的预期目标主要包括：

1. 从协议参数角度系统总结 OAuth 实现中的常见安全攻击类型，明确其产生原因及现实影响；
2. 指出 MCP 原生授权模型在使用 OAuth 过程中存在的结构性安全问题；
3. 设计一种以 Gateway 为核心的 MCP-AuthZ 授权增强架构，并给出完整的授权流程与机制说明；
4. 从安全性与工程可行性角度论证该架构在提升 MCP 授权安全性方面的有效性。

通过上述研究，本文期望为 OAuth 在新型协议与系统中的安全应用提供系统化分析方法与可行的设计思路，并为 MCP 等模型工具调用框架的安全演进提供参考。

四 研究进展和阶段性成果

围绕 OAuth 协议实现安全性分析及其在 Model Context Protocol (MCP) 授权场景中的应用问题，本文在中期阶段已完成从理论分析、协议设计到工程原型实现的系统性研究工作。当前阶段性成果不仅体现在对 OAuth 与 MCP 授权安全问题的深入分析上，也体现在 MCP-AuthZ Gateway 的实际工程实现与验证上。

4.1 OAuth 实现安全分析的阶段性研究成果

在理论分析层面，本文已完成对 OAuth 2.0 协议实现中常见攻击类型的系统化整理与建模。通过以协议参数及其安全语义为核心分析维度，本文对 CSRF、授权码拦截、重定向 URI 攻击、客户端冒充及访问令牌滥用等问题进行了统一抽象，明确了这些攻击在本质上均源于授权上下文绑定不充分或参数安全语义在实现层面的弱化。

该分析过程为后续研究 OAuth 在 MCP 场景中的适用性提供了方法论基础，也为 Gateway 设计中“需要显式执行哪些安全约束”给出了明确依据。

4.2 MCP 原生授权流程安全问题的分析进展

在对 MCP 协议进行深入分析后，本文指出 MCP 原生设计中将 OAuth 作为“外部授权机制”引入，但并未在协议层对授权结果的使用方式进行系统建模。在实际调用链中，OAuth 授权请求通常由 MCP Client 发起，而访问令牌的使用主体却是 MCP Server 或其后端工具服务，这导致 OAuth 中“客户端即资源使用者”的核心安全假设在 MCP 场景下不再成立。

通过将 MCP 原生授权流程与已知 OAuth 攻击模型进行对照分析，本文进一步确认：在缺乏额外授权控制层的情况下，MCP 极易受到 confused deputy、权限扩散、令牌重放等攻击。这一分析结论直接推动了 MCP-AuthZ Gateway 方案的提出。

4.3 MCP-AuthZ Gateway 的协议与架构设计成果

针对 MCP 原生授权模型的不足，本文提出并完成了 MCP-AuthZ 架构设计，引入独立的 MCP-Auth Gateway 作为授权与安全控制的集中执行点。Gateway 位于 MCP Client 与 MCP Server 之间，负责对 OAuth 授权结果进行解释、约束与执行，而非简单转发访问令牌。

在协议层面，本文已完成以下关键设计：

- 引入 Signed Manifest 机制，对工具身份、能力范围及安全级别进行加密签名与验证；
- 将工具能力抽象为 A/B/C 三类 profile，并将 profile 作为授权协商与访问控制的一等协议要素；
- 设计基于 OAuth 的 scope negotiation 机制，使授权结果与具体工具能力精确匹配；
- 在调用阶段引入请求级签名与 token binding (PoP) 机制，防止令牌滥用与重放；

- 设计授权撤销与状态查询接口，为 MCP 授权引入完整生命周期管理能力。

上述设计已形成较为完整的 MCP-AuthZ 协议框架，并具备工程实现条件。

4.4 MCP-Auth Gateway 的工程实现进展

在工程实现层面，本文已完成 MCP-Auth Gateway 的初步原型系统，并在开源仓库中给出了实现代码¹。该原型系统采用模块化设计，实现了 MCP-AuthZ 架构中的关键功能组件。

目前已实现的功能包括：

- **Gateway 基础转发能力**：作为 MCP Client 与 MCP Server 之间的中介组件，支持兼容转发原生 MCP 消息；
- **工具初始化阶段的安全增强**：在 initialize 阶段引入对工具 manifest 的获取与校验逻辑，为后续授权与调用建立可信基础；
- **授权信息管理模块**：支持对 OAuth 授权结果的接收、解析与内部表示，为调用阶段的权限判定提供输入；
- **调用前权限检查机制**：在工具调用请求进入 MCP Server 前，基于工具标识与授权上下文执行访问控制决策。

上述实现验证了 MCP-AuthZ Gateway 在不修改原生 MCP Server 的前提下引入授权控制层的可行性，也为后续安全机制（如 PoP 校验、请求级签名、撤销接口等）的逐步集成提供了稳定基础。

4.5 阶段性评估设计与后续实现方向

在当前实现基础上，本文已完成对实验评估方案的初步设计。后续工作将围绕以下方向展开：

- **安全性评估**：构造典型 OAuth 攻击与 MCP 授权滥用场景，对比原生 MCP 与 MCP-AuthZ 在防护效果上的差异；
- **兼容性评估**：验证 Gateway 在不破坏 MCP 现有接口与工具实现的情况下对授权流程的增强能力；
- **性能评估**：测量 Gateway 引入后的请求延迟与系统开销，分析安全增强带来的性能影响。

¹<https://github.com/quietfreedom/mcp-auth-gateway>

4.6 阶段性成果总结

综上，本文在中期阶段已完成从 OAuth 安全问题分析、MCP 授权模型缺陷识别，到 MCP-AuthZ 协议设计与 Gateway 原型实现的完整研究链路。相关成果表明，本文已从概念性研究推进至可实现、可验证的系统设计阶段，为后续实验评估与论文定稿奠定了坚实基础。

五 研究工作计划与进度安排

阶段	研究任务	时间安排
阶段一	完成 Gateway 工程实现，并进行评估实验	2025 年 12 月-2026 年 1 月
阶段二	完成实验数据整理，总结研究成果，撰写项目总结和论文大纲	2026 年 1 月-2026 年 2 月
阶段三	完成论文撰写，准备答辩	2026 年 2 月-2026 年 4 月

参考文献

- [1] Hsu C, Lin J C C. Acceptance of blog usage: The roles of technology acceptance, social influence and knowledge sharing motivation [J]. Information & Management, 2008, 45(1): 65-74.
- [2] Bansal C, Bhargavan K, Maffei S. Discovering concrete attacks on website authorization by formal analysis [C/OL]//Proceedings of the 25th IEEE Computer Security Foundations Symposium (CSF). IEEE, 2012: 247-262. <https://doi.org/10.1109/CSF.2012.27>.
- [3] Fett D, Küsters R, Schmitz G. A comprehensive formal security analysis of oauth 2.0 [C/OL]//Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS). ACM, 2016: 1204-1215. <https://doi.org/10.1145/2976749.2978385>.
- [4] Andrade R, Yoo S, Tello-Oquendo L, et al. A comprehensive study of the iot cybersecurity in smart cities [J/OL]. IEEE Access, 2020, 8: 228922-228941. <https://doi.org/10.1109/ACCESS.2020.3046442>.
- [5] Sucasas V, Mantas G, Althunibat S, et al. A privacy-enhanced oauth 2.0 based protocol for smart city mobile applications [J/OL]. Computers & Security, 2018, 74: 258-274. <https://doi.org/10.1016/j.cose.2018.01.014>.

- [6] Microsoft azure account takeover [EB/OL]. 2020. <https://www.cyberark.com/resources/threat-research-blog/blackdirect-microsoft-azure-account-takeover>.
- [7] Facebook security update [EB/OL]. 2018. <https://about.fb.com/news/2018/09/security-update/>.
- [8] Ferry E, O’Raw J, Curran K. Security evaluation of the oauth 2.0 framework [J/OL]. Information and Computer Security, 2015, 23(1): 73-101. <https://doi.org/10.1108/ICS-12-2013-0089>.
- [9] Sun S, Beznosov K. The devil is in the (implementation) details: An empirical analysis of oauth sso systems [C/OL]//Proceedings of the 2012 ACM Conference on Computer and Communications Security (CCS). ACM, 2012: 378-390. <https://doi.org/10.1145/2382196.2382238>.
- [10] Alotaibi A, Mahmmud A. Enhancing oauth services security by an authentication service with face recognition [C/OL]//Proceedings of the 11th Long Island Systems, Applications and Technology Conference (LISAT). IEEE, 2015. <https://doi.org/10.1109/LISAT.2015.7160208>.
- [11] Dabagh N, Mahmood M. Multilevel database security for android using fast encryption methods [J/OL]. Al-Rafidain Journal of Computer Sciences and Mathematics, 2022, 16(1): 87-96. <https://doi.org/10.33899/csmj.2022.174412>.
- [12] Li W, Mitchell C. Your code is my code: Exploiting a common weakness in oauth 2.0 implementations [M/OL]//Bodden E, Athanasopoulos E, Monroe F. Lecture Notes in Computer Science: volume 10885 Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2018). Springer, 2018: 24-41. https://doi.org/10.1007/978-3-030-03251-7_3.
- [13] Li W, Mitchell C, Chen T. Oauthguard [C/OL]//Proceedings of the 35th Annual Computer Security Applications Conference (ACSAC). ACM, 2019: 444-456. <https://doi.org/10.1145/3338500.3360331>.
- [14] Arshad E, Benolli M, Crispo B. Practical attacks on login csrf in oauth [J/OL]. Computers & Security, 2022, 121: 102859. <https://doi.org/10.1016/j.cose.2022.102859>.
- [15] Innocenti T, Golinelli M, Onarlioglu K, et al. Oauth 2.0 redirect uri validation falls short, literally [C/OL]//Proceedings of the 2023 Annual Computer Security Applications Conference (ACSAC ’ 23). Austin, TX, USA: ACM, 2023: xx-xx. <https://doi.org/10.1145/3627106.3627140>.
- [16] Bhattacharya S, Najana M, Khanna A, et al. Securing the gatekeeper: Addressing vulnerabilities in oauth implementations for enhanced web security [J/OL]. International Journal of Global Innovations and Solutions (IJGIS), 2024, April 2024. <https://ijgis.pubpub.org/pub/jkavqi25>. DOI: [10.21428/e90189c8.af381673](https://doi.org/10.21428/e90189c8.af381673).
- [17] Shernan E, Carter H, Tian D, et al. More guidelines than rules: Csrft vulnerabilities from noncompliant oauth 2.0 implementations [C/OL]//Lecture Notes in Computer Science: volume 9148 Proceedings of the 2015 International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2015). Springer, Cham, 2015: 239-260. https://doi.org/10.1007/978-3-319-20550-2_13.
- [18] Li W, Mitchell C J. Security issues in oauth 2.0 sso implementations [C/OL]//Lecture Notes in Com-

- puter Science: volume 8783 International Conference on Information Security (ISC 2014). Springer, Cham, 2014: 529-541. https://doi.org/10.1007/978-3-319-13257-0_34.
- [19] Sudhodanan A, Carbone R, Compagna L, et al. Large-scale analysis & detection of authentication cross-site request forgeries [C/OL]//Proceedings of the 2017 IEEE European Symposium on Security and Privacy (EuroS&P). Paris, France: IEEE, 2017: 350-365. <https://doi.org/10.1109/EUROSP.2017.45>.
 - [20] Nettleship W, Campbell B, Bradley J, et al. Proof key for code exchange by oauth public clients [M/OL]. Internet Engineering Task Force, 2015. <https://www.rfc-editor.org/info/rfc7636>. DOI: 10.17487/RFC7636.
 - [21] Parecki A, Hardt D. Oauth 2.1 [EB/OL]. Internet Engineering Task Force, 2023. <https://datatracker.ietf.org/doc/html/draft-ietf-oauth-v2-1-10>.
 - [22] Barth A, Jackson C, Mitchell J C. Robust defenses for cross-site request forgery [C/OL]//Proceedings of the 15th ACM Conference on Computer and Communications Security (CCS ' 08). Alexandria, VA, USA: ACM, 2008: 75-88. <https://seclab.stanford.edu/websec/csrf/csrf.pdf>. DOI: 10.1145/1455770.1455784.
 - [23] Salt Labs, Salt Security. Oh-auth—abusing oauth to take over millions of accounts [EB/OL]. 2023. <https://salt.security/blog/oh-auth-abusing-oauth-to-take-over-millions-of-accounts>.
 - [24] Singh J, Chaudhary N K. Unified singular protocol flow for oauth (uspfo) ecosystem [J/OL]. arXiv preprint arXiv:2301.12496, 2023. <https://arxiv.org/abs/2301.12496>. DOI: 10.48550/arXiv.2301.12496.
 - [25] Anthropic. Model context protocol specification [EB/OL]. 2024. <https://modelcontextprotocol.io>.
 - [26] Zou A, et al. Tool use and security risks in large language models [J]. arXiv preprint arXiv:2307.15043, 2023.
 - [27] Schick T, Dwivedi-Yu J, et al. Toolformer: Language models can teach themselves to use tools [J]. arXiv preprint arXiv:2302.04761, 2023.
 - [28] OpenAI. Security best practices for llm tool usage [Z]. 2024.
 - [29] Anthropic. Security considerations for tool-enabled language models [Z]. 2024.
 - [30] Hardt D. Request for comments: number 6749 The OAuth 2.0 Authorization Framework [M/OL]. RFC Editor, 2012. <https://www.rfc-editor.org/info/rfc6749>. DOI: 10.17487/RFC6749.
 - [31] Sakimura N, Bradley J, Jones M, et al. OpenID Connect Core 1.0 incorporating Errata Set 2 [EB/OL]. 2014. https://openid.net/specs/openid-connect-core-1_0.html.
 - [32] Fett D, Küsters R, Schmitz G. The web sso standard openid connect: In-depth formal security analysis and security guidelines [C/OL]//Proceedings of the 30th IEEE Computer Security Foundations Symposium (CSF). IEEE, 2017: 189-202. <https://doi.org/10.1109/CSF.2017.20>.
 - [33] Philippaerts P, Preuveneers D, Joosen W. Oauch: Exploring security compliance in the oauth 2.0

- ecosystem [C/OL]//Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing (SAC). ACM, 2022: 1844-1853. <https://doi.org/10.1145/3545948.3545955>.
- [34] Munonye K, Martinek P. Machine learning approach to vulnerability detection in oauth 2.0 authentication and authorization flow [J/OL]. International Journal of Information Security, 2021, 21(2): 223-237. <https://doi.org/10.1007/s10207-021-00551-w>.
- [35] Azizul N, Mohd A, Chandren R, et al. Authentication and authorization design in honeybee computing [J/OL]. International Journal of Advanced Computer Science and Applications (IJACSA), 2019, 10(9). <https://doi.org/10.14569/ijacsa.2019.0100903>. DOI: 10.14569/IJACSA.2019.0100903.
- [36] Ismail U, Islam S, Ouedraogo M, et al. A framework for security transparency in cloud computing [J/OL]. Future Internet, 2016, 8(1): 5. <https://doi.org/10.3390/fi8010005>.
- [37] Al Rahat T, Feng Y, Tian Y. Cerberus: Query-driven scalable vulnerability detection in oauth service provider implementations [C/OL]//CCS: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS ' 22). Los Angeles, CA, USA: ACM, 2022: 1-15. <https://doi.org/10.1145/3548606.3559381>.
- [38] Yang R, Li G, Lau W C, et al. OAuthTester: Model-based security testing of oauth 2.0 implementations [C/OL]//Proceedings of the 2016 ACM Asia Conference on Computer and Communications Security (ASIACCS '16). Xi'an, China: ACM, 2016. <https://doi.org/10.1145/2897845.2897874>.
- [39] Westers M, Wich T, Jannett L, et al. Sso-monitor: Fully-automatic large-scale landscape, security, and privacy analyses of single sign-on in the wild [J/OL]. arXiv preprint arXiv:2302.01024, 2023. <https://arxiv.org/abs/2302.01024>.